2026-05-05-p1-f03-tool-result-persistence

P1-F03 — Tool Result Persistence Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Implement claude-code-style tool-result persistence: when a tool produces output exceeding 50,000 characters, save it to `<cwd>/.helix/tool-results/` and substitute a path-reference in the LLM payload. The LLM reads back via the existing `Read` tool. A 7-day age-based sweep runs lazily at startup.

Architecture: New thin sub-package `internal/tools/persistence/` (mirrors F02's `internal/tools/permissions/`). Threshold check fires at the LLM provider boundary in `internal/llm/tool_provider.go` — the orchestration loop where `executeToolCalls` produces results before they're handed to `buildFinalPrompt`. A `Manager` is constructed once at CLI startup and injected into the `ToolCallingProvider`. System-prompt note teaches the LLM the convention.

Tech Stack: Go 1.26, testify v1.11, existing `internal/tools/` packages. **No new dependencies.** Standard-library `crypto/sha256`, `encoding/hex`, `os`, `path/filepath`, `time`, `sync` only.

Spec: `docs/superpowers/specs/2026-05-05-p1-f03-tool-result-persistence-design.md` (commit `f813fc9`)

Working directory for all `go` commands: `helix_code/` (the inner Go module). Git commands run from the meta-repo root `/run/media/milosvasic/DATA4TB/Projects/helix_code/` per the F01/F02 convention.

Anti-bluff smoke (run on every commit):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/tools/persistence/ && echo "BLUFF
        FOUND" || echo "clean"
```

Task 1: Bootstrap evidence + advance PROGRESS

Files: - Modify: `docs/improvements/06_phase_1_evidence.md` - Modify: `docs/improvements/PROGRESS.md`

- [ ] **Step 1: Append F03 section header to evidence file**

Append to `docs/improvements/06_phase_1_evidence.md`:

```
---

## P1-F03 — Tool Result Persistence

**Spec:** `docs/superpowers/specs/2026-05-05-p1-f03-tool-result-
        persistence-design.md` (commit `f813fc9`)
**Plan:** `docs/superpowers/plans/2026-05-05-p1-f03-tool-result-
        persistence.md`
**Started:** 2026-05-05
**Status:** active

### Task evidence trail

(filled in commit-by-commit as tasks land)
```

☐ **Step 2: Update PROGRESS.md current focus block**

Replace the existing "## Current focus" block in `docs/improvements/PROGRESS.md` with:

```
## Current focus
- **Active phase:** P1 — claude-code feature porting
- **Active feature:** F03 — Tool Result Persistence
- **Active task:** P1-F03-T01 — bootstrap evidence + advance
        PROGRESS
- **Last completed:** P1-F02-T13 — Feature 2 (Permission Rule
        System) close-out + push
- **Owner:** agent (Claude Opus 4.7)
- **Started:** 2026-05-04
- **Last touched:** 2026-05-05
- **Blocked-on:** none
```

☐ **Step 3: Add F03 task list block to PROGRESS.md**

After the existing F02 task list block, insert:

```
## Active feature task list (P1-F03: Tool Result Persistence)
- [ ] P1-F03-T01 — bootstrap evidence + advance PROGRESS
- [ ] P1-F03-T02 — internal/tools/persistence package skeleton
        (types + doc)
- [ ] P1-F03-T03 — Manager.MaybePersist with hash idempotence
        (TDD)
- [ ] P1-F03-T04 — LoadPersisted with path-traversal guard (TDD)
- [ ] P1-F03-T05 — CleanupOld with filename-pattern matching
        (TDD)
- [ ] P1-F03-T06 — wire into internal/llm/tool_provider.go
        orchestration loop
```

```
- [ ] P1-F03-T07 — audit + wire individual LLM providers
- [ ] P1-F03-T08 — system prompt note about persistedOutputPath
-
        [ ] P1-F03-T09 — cmd/cli/main.go startup + integration
        test (no mocks)
- [ ] P1-F03-T10 — Challenge with three runtime-evidence
        scenarios
- [ ] P1-F03-T11 — Feature 3 close-out + push
```

☐ **Step 4: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
        improvements/06_phase_1_evidence.md docs/improvements/
        PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
docs(P1-F03-T01): bootstrap Phase 1 / Feature 3 evidence +
        advance PROGRESS

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 2: Permissions package skeleton (types + doc)

Files: - Create: `helix_code/internal/tools/persistence/doc.go` - Create: `helix_code/internal/tools/persistence/types.go`

☐ **Step 1: Create doc.go**

Create `helix_code/internal/tools/persistence/doc.go`:

```
// Package persistence implements claude-code-style tool-result
        persistence.
//
// When a tool's output exceeds PersistThreshold characters, the
        runtime
// writes the raw content to <projectRoot>/.helix/tool-results/
        and the
// LLM payload carries a path-reference instead of inline
        content. The
// LLM reads back via the existing Read tool. A 7-day age-based
        sweep
// runs lazily at startup via Manager.CleanupOld.
//
```

```
// See: docs/superpowers/specs/2026-05-05-p1-f03-tool-result-
        persistence-design.md
package persistence
```

☐ **Step 2: Create types.go**

Create `helix_code/internal/tools/persistence/types.go`:

```
package persistence

import "time"

// Constants control persistence threshold, on-disk location, and
        cleanup window.
const (
    // PersistThreshold is the byte count above which outputs are
        persisted.
    // A result with len([]byte(output)) > PersistThreshold is
        persisted.
    // A result with len(...) == PersistThreshold stays inline
        (boundary is
    // strictly greater than).
    PersistThreshold = 50_000

    // PersistDir is the relative path under projectRoot for
        persisted outputs.
    PersistDir = ".helix/tool-results"

    // DefaultMaxAge is the default cleanup window for
        CleanupOld.
    DefaultMaxAge = 7 * 24 * time.Hour
)

// PersistedResult represents a tool result, either inline or
        persisted-to-disk.
//
// Output and PersistedOutputPath are mutually exclusive.
        WasPersisted is
// the canonical boolean — providers serialise this struct to a
        wire format
// by branching on WasPersisted.
type PersistedResult struct {
    Output              string
        `json:"output,omitempty"`              // empty if
        persisted
    PersistedOutputPath string
        `json:"persistedOutputPath,omitempty"` // absolute path
        on disk
```

```
    PersistedOutputSize int
        `json:"persistedOutputSize,omitempty"` // byte count of
        the original content
    WasPersisted        bool   `json:"wasPersisted"`
    ToolName            string `json:"toolName"`
    ToolCallID          string `json:"toolCallID,omitempty"`
}
```

- [ ] **Step 3: Verify the package compiles**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go build ./internal/tools/persistence/...
```

Expected: clean compile, exit 0.

- [ ] **Step 4: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/tools/persistence/ && echo "BLUFF
        FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 5: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/tools/persistence/doc.go helix_code/
        internal/tools/persistence/types.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T02): add internal/tools/persistence package skeleton

Doc.go declares package purpose. Types.go defines PersistedResult
        struct
+ constants (PersistThreshold = 50_000 bytes, PersistDir =
        ".helix/tool-results",
DefaultMaxAge = 7d). Compiles clean; anti-bluff smoke clean.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 3: Manager.MaybePersist with hash idempotence (TDD)

Files: - Create: `helix_code/internal/tools/persistence/manager.go` - Create: `helix_code/internal/tools/persistence/manager_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/tools/persistence/manager_test.go`:

```
package persistence

import (
    "crypto/sha256"
    "encoding/hex"
    "os"
    "path/filepath"
    "strings"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestMaybePersist_BelowThresholdIsInline(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    output := strings.Repeat("X", PersistThreshold-1)
    res, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err)
    assert.False(t, res.WasPersisted)
    assert.Equal(t, output, res.Output)
    assert.Empty(t, res.PersistedOutputPath)
    assert.Equal(t, "Bash", res.ToolName)
    assert.Equal(t, "call-1", res.ToolCallID)
}

func TestMaybePersist_AtThresholdIsInline(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    output := strings.Repeat("X", PersistThreshold)
    res, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err)
    assert.False(t, res.WasPersisted, "len == PersistThreshold
        must stay inline (boundary strictly greater)")
    assert.Equal(t, output, res.Output)
}
```

```
func TestMaybePersist_AboveThresholdIsPersisted(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    output := strings.Repeat("X", PersistThreshold+1)
    res, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err)
    assert.True(t, res.WasPersisted)
    assert.Empty(t, res.Output)
    assert.NotEmpty(t, res.PersistedOutputPath)
    assert.Equal(t, PersistThreshold+1, res.PersistedOutputSize)

    body, err := os.ReadFile(res.PersistedOutputPath)
    require.NoError(t, err)
    assert.Equal(t, output, string(body))
}

func TestMaybePersist_HashIdempotence(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    output := strings.Repeat("Y", PersistThreshold+10)

    r1, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err)
    require.True(t, r1.WasPersisted)

    r2, err := m.MaybePersist("Bash", "call-2", output)
    require.NoError(t, err)
    require.True(t, r2.WasPersisted)

    // Hash idempotence: same content + same tool produces the
        same filename
    // (timestamp differs but only at second-granularity; the
        hash collision
    // in filename means same path on subsequent persistence).
    hash := sha256.Sum256([]byte(output))
    expectedHashPrefix := hex.EncodeToString(hash[:8])
    assert.Contains(t, r1.PersistedOutputPath,
        expectedHashPrefix,
        "filename must include sha256[:16] of content")
    assert.Contains(t, r2.PersistedOutputPath,
        expectedHashPrefix)
}

func TestMaybePersist_FilenameSanitises(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    output := strings.Repeat("Z", PersistThreshold+1)
```

```
    res, err := m.MaybePersist("../etc/passwd", "call-1", output)
    require.NoError(t, err)
    require.True(t, res.WasPersisted)

    // Filename must not contain "..", "/", or "\"
    base := filepath.Base(res.PersistedOutputPath)
    assert.NotContains(t, base, "..")
    assert.NotContains(t, base, "/")
    assert.NotContains(t, base, "\\")

    // File still lands inside the persistence dir
    dir := filepath.Dir(res.PersistedOutputPath)
    expectedBase := filepath.Join(tmp, PersistDir)
    assert.Equal(t, expectedBase, dir)
}

func TestMaybePersist_EmptyOutputIsInline(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    res, err := m.MaybePersist("Bash", "call-1", "")
    require.NoError(t, err)
    assert.False(t, res.WasPersisted)
    assert.Empty(t, res.Output)
    assert.Equal(t, "Bash", res.ToolName)
}

func TestMaybePersist_NilManagerIsSafe(t *testing.T) {
    // Defensive: a nil *Manager must not panic; treat as
        persistence disabled.
    var m *Manager
    res, err := m.MaybePersist("Bash", "call-1",
        strings.Repeat("X", PersistThreshold+1))
    require.NoError(t, err)
    assert.False(t, res.WasPersisted,
        "nil Manager passes through inline")
    assert.Equal(t, strings.Repeat("X", PersistThreshold+1),
        res.Output)
}

func TestMaybePersist_DiskFullFallsBackToInline(t *testing.T) {
    if os.Getuid() == 0 {
        t.Skip("SKIP-OK: #permissions-as-root — root bypasses
        chmod 0500 in this test")
    }
    tmp := t.TempDir()
    m := NewManager(tmp)
```

```
    // Pre-create the dir read-only so MaybePersist's WriteFile
        fails.
    require.NoError(t, os.MkdirAll(filepath.Join(tmp,
        PersistDir), 0o500))

    output := strings.Repeat("X", PersistThreshold+1)
    res, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err, "disk-full must NOT propagate as
        error — fall back to inline")
    assert.False(t, res.WasPersisted)
    assert.Equal(t, output, res.Output)
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run TestMaybePersist ./internal/
        tools/persistence/
```

Expected: FAIL — `Manager`, `NewManager`, `MaybePersist` undefined.

- [ ] **Step 3: Implement manager.go**

Create `helix_code/internal/tools/persistence/manager.go`:

```
package persistence

import (
    "crypto/sha256"
    "encoding/hex"
    "errors"
    "fmt"
    "log"
    "os"
    "path/filepath"
    "regexp"
    "strings"
    "sync"
    "time"
)

// Manager owns a project's tool-result blob store.
type Manager struct {
    baseDir string       // <projectRoot>/<PersistDir>
    mu      sync.RWMutex // RLock for LoadPersisted; WLock for
        MaybePersist + CleanupOld
}
```

```
// NewManager returns a Manager rooted at projectRoot. The
        persistence dir
// is created lazily on the first persist; calling NewManager
        does NOT do I/O.
func NewManager(projectRoot string) *Manager {
    return &Manager{
        baseDir: filepath.Join(projectRoot, PersistDir),
    }
}

// MaybePersist returns an inline PersistedResult for output
        sized
// <= PersistThreshold; otherwise writes the content to disk and
        returns a
// path-reference. A nil *Manager passes through inline.
//
// Disk failures fall back to inline (logged at WARN); the tool
        call's
// downstream visibility is preserved at the cost of LLM token
        usage.
func (m *Manager) MaybePersist(toolName, toolCallID, output
        string) (*PersistedResult, error) {
    if m == nil {
        return &PersistedResult{
            Output:     output,
            ToolName:   toolName,
            ToolCallID: toolCallID,
        }, nil
    }

    if len(output) <= PersistThreshold {
        return &PersistedResult{
            Output:     output,
            ToolName:   toolName,
            ToolCallID: toolCallID,
        }, nil
    }

    m.mu.Lock()
    defer m.mu.Unlock()

    if err := os.MkdirAll(m.baseDir, 0o755); err != nil {

        log.Printf("WARN persistence: mkdir %s: %v — falling back
        to inline", m.baseDir, err)
        return &PersistedResult{
            Output:     output,
            ToolName:   toolName,
```

```
            ToolCallID: toolCallID,
        }, nil
    }

    hash := sha256.Sum256([]byte(output))
    hashHex := hex.EncodeToString(hash[:8]) // 16 hex chars
    timestamp := time.Now().UTC().Format("20060102_150405")
    filename := fmt.Sprintf("%s_%s_%s.txt",
        sanitiseToolName(toolName), hashHex, timestamp)
    path := filepath.Join(m.baseDir, filename)

    if err := os.WriteFile(path, []byte(output), 0o644); err !=
        nil {

        log.Printf("WARN persistence: write %s: %v — falling back
        to inline", path, err)
        return &PersistedResult{
            Output:     output,
            ToolName:   toolName,
            ToolCallID: toolCallID,
        }, nil
    }

    return &PersistedResult{
        PersistedOutputPath: path,
        PersistedOutputSize: len(output),
        WasPersisted:        true,
        ToolName:            toolName,
        ToolCallID:          toolCallID,
    }, nil
}

// sanitiseToolName strips path separators, traversal, control
        characters,
// and clamps to 32 chars so the filename is filesystem-safe.
var safeNameRune = regexp.MustCompile(`[^A-Za-z0-9._-]`)

func sanitiseToolName(name string) string {
    cleaned := safeNameRune.ReplaceAllString(name, "_")
    cleaned = strings.ReplaceAll(cleaned, "..", "__")
    if len(cleaned) > 32 {
        cleaned = cleaned[:32]
    }
    if cleaned == "" {
        cleaned = "tool"
    }
    return cleaned
}
```

```
// ErrPathTraversal is returned by LoadPersisted when the
        requested path
// resolves outside the manager's base directory.
var ErrPathTraversal = errors.New("path outside persistence
        directory")
```

- [ ] **Step 4: Run tests to verify they pass**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run TestMaybePersist ./
        internal/tools/persistence/
```

Expected: PASS for all 8 named tests.

- [ ] **Step 5: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/tools/persistence/ && echo "BLUFF
        FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 6: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/tools/persistence/manager.go
        helix_code/internal/tools/persistence/manager_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T03): Manager.MaybePersist with hash idempotence
        (TDD)

Manager rooted at <projectRoot>; lazy mkdir on first persist.
        Outputs
exceeding PersistThreshold (50_000 bytes) are written to disk
        under
.helix/tool-results/ with a sha256[:16]-hashed filename. Disk
        failures
fall back to inline + WARN log. Nil Manager safely passes
        through.
8 unit tests with -race: below/at/above threshold, hash
        idempotence,
filename sanitisation (../etc/passwd input safely contained),
empty-output, nil-receiver, disk-full fallback.
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 4: LoadPersisted with path-traversal guard (TDD)

Files: - Modify: `helix_code/internal/tools/persistence/manager.go` (add `LoadPersisted`) - Create: `helix_code/internal/tools/persistence/load_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/tools/persistence/load_test.go`:

```
package persistence

import (
    "errors"
    "os"
    "path/filepath"
    "strings"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestLoadPersisted_HappyPath(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)

    output := strings.Repeat("A", PersistThreshold+5)
    res, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err)
    require.True(t, res.WasPersisted)

    got, err := m.LoadPersisted(res.PersistedOutputPath)
    require.NoError(t, err)
    assert.Equal(t, output, got)
}

func TestLoadPersisted_RejectsParentTraversal(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)

    // Pre-create a sensitive file outside the persistence dir
```

```
    sensitive := filepath.Join(tmp, "secrets.txt")
    require.NoError(t, os.WriteFile(sensitive,
        []byte("topsecret"), 0o600))

    // Try to load via a relative-traversal path
    traversal := filepath.Join(tmp, PersistDir, "..",
        "secrets.txt")
    _, err := m.LoadPersisted(traversal)
    require.Error(t, err)
    assert.True(t, errors.Is(err, ErrPathTraversal),
        "expected ErrPathTraversal, got %v", err)
}

func TestLoadPersisted_RejectsAbsoluteOutsideBase(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)

    // Pre-create a sensitive file in tmp (outside the
        persistence dir)
    sensitive := filepath.Join(tmp, "etc_passwd")
    require.NoError(t, os.WriteFile(sensitive, []byte("uid=0"),
        0o600))

    _, err := m.LoadPersisted(sensitive)
    require.Error(t, err)
    assert.True(t, errors.Is(err, ErrPathTraversal),
        "absolute path outside base must be rejected with
        ErrPathTraversal, got %v", err)
}

func TestLoadPersisted_MissingFileWrapsErrNotExist(t *testing.T)
        {
    tmp := t.TempDir()
    m := NewManager(tmp)

    missing := filepath.Join(tmp, PersistDir, "nope.txt")
    _, err := m.LoadPersisted(missing)
    require.Error(t, err)
    assert.True(t, errors.Is(err, os.ErrNotExist),
        "missing file must wrap os.ErrNotExist, got %v", err)
}
```

☐ **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run TestLoadPersisted ./internal/
        tools/persistence/
```

Expected: FAIL — `LoadPersisted` undefined.

- [ ] **Step 3: Add LoadPersisted to manager.go**

Append to `helix_code/internal/tools/persistence/manager.go`:

```
// LoadPersisted reads a previously-persisted output by absolute
        path.
// Returns ErrPathTraversal if path resolves outside the
        manager's base
// directory; wraps os.ErrNotExist for missing files.
func (m *Manager) LoadPersisted(path string) (string, error) {
    m.mu.RLock()
    defer m.mu.RUnlock()

    absPath, err := filepath.Abs(path)
    if err != nil {
        return "", fmt.Errorf("resolving %s: %w", path, err)
    }
    absBase, err := filepath.Abs(m.baseDir)
    if err != nil {
        return "", fmt.Errorf("resolving base %s: %w",
        m.baseDir, err)
    }
    rel, err := filepath.Rel(absBase, absPath)
    if err != nil || strings.HasPrefix(rel, "..") || rel == ".."
        {
        return "", fmt.Errorf("%w: %s", ErrPathTraversal, path)
    }

    body, err := os.ReadFile(absPath)
    if err != nil {
        return "", fmt.Errorf("reading %s: %w", absPath, err)
    }
    return string(body), nil
}
```

(`strings`, `fmt`, `os`, `filepath` are already imported by manager.go from T03; no new imports needed.)

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestLoadPersisted|
        TestMaybePersist' ./internal/tools/persistence/
```

Expected: PASS for all 4 LoadPersisted tests + the 8 MaybePersist tests from T03.

- [ ] **Step 5: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/tools/persistence/ && echo "BLUFF
        FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 6: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/tools/persistence/manager.go
        helix_code/internal/tools/persistence/load_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T04): LoadPersisted with path-traversal guard (TDD)

Adds Manager.LoadPersisted that uses filepath.Rel against the
        resolved
base dir to reject any path outside .helix/tool-results/. Returns
ErrPathTraversal for traversal attempts (..-relative or absolute
        outside
base) and wraps os.ErrNotExist for missing files. 4 unit tests
        including
the parent-traversal vector and absolute-path-to-secrets attempt.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 5: CleanupOld with filename-pattern matching (TDD)

Files: - Modify: `helix_code/internal/tools/persistence/manager.go` (add `CleanupOld`) - Create: `helix_code/internal/tools/persistence/cleanup_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/tools/persistence/cleanup_test.go`:

```
package persistence
```

```
import (
    "os"
    "path/filepath"
    "strings"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestCleanupOld_RemovesAgedFiles(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    require.NoError(t, os.MkdirAll(m.baseDir, 0o755))

    old := filepath.Join(m.baseDir,
        "Bash_aabbccddeeff0011_20200101_000000.txt")
    require.NoError(t, os.WriteFile(old,
        []byte(strings.Repeat("X", 100)), 0o644))
    require.NoError(t, os.Chtimes(old,
        time.Now().Add(-30*24*time.Hour),
        time.Now().Add(-30*24*time.Hour)))

    new := filepath.Join(m.baseDir,
        "Bash_001122334455_20260101_000000.txt")
    require.NoError(t, os.WriteFile(new,
        []byte(strings.Repeat("Y", 100)), 0o644))

    require.NoError(t, m.CleanupOld(7*24*time.Hour))

    _, errOld := os.Stat(old)
    assert.True(t, os.IsNotExist(errOld), "aged file must be
        deleted")
    _, errNew := os.Stat(new)
    assert.NoError(t, errNew, "fresh file must remain")
}

func TestCleanupOld_LeavesNonPatternFiles(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    require.NoError(t, os.MkdirAll(m.baseDir, 0o755))

    gitkeep := filepath.Join(m.baseDir, ".gitkeep")
    require.NoError(t, os.WriteFile(gitkeep, []byte(""), 0o644))
    require.NoError(t, os.Chtimes(gitkeep,
        time.Now().Add(-30*24*time.Hour),
        time.Now().Add(-30*24*time.Hour)))
```

```
    readme := filepath.Join(m.baseDir, "README.md")
    require.NoError(t, os.WriteFile(readme, []byte("docs"),
        0o644))
    require.NoError(t, os.Chtimes(readme,
        time.Now().Add(-30*24*time.Hour),
        time.Now().Add(-30*24*time.Hour)))

    require.NoError(t, m.CleanupOld(7*24*time.Hour))

    _, errKeep := os.Stat(gitkeep)
    assert.NoError(t, errKeep, ".gitkeep must be left alone")
    _, errReadme := os.Stat(readme)
    assert.NoError(t, errReadme, "README.md must be left alone")
}

func TestCleanupOld_LeavesDirectories(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp)
    require.NoError(t, os.MkdirAll(filepath.Join(m.baseDir,
        "subdir"), 0o755))

    require.NoError(t, m.CleanupOld(7*24*time.Hour))

    info, err := os.Stat(filepath.Join(m.baseDir, "subdir"))
    require.NoError(t, err)
    assert.True(t, info.IsDir())
}

func TestCleanupOld_MissingBaseDirIsNoOp(t *testing.T) {
    tmp := t.TempDir()
    m := NewManager(tmp) // baseDir not created
    require.NoError(t, m.CleanupOld(7*24*time.Hour))
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run TestCleanupOld ./internal/tools/
        persistence/
```

Expected: FAIL — `CleanupOld` undefined.

- [ ] **Step 3: Add CleanupOld to manager.go**

Append to `helix_code/internal/tools/persistence/manager.go`:

```
// filenamePattern matches the canonical persistence filename
        format:
//   <sanitised-tool>_<16-hex-hash>_<UTC-yyyymmdd_hhmmss>.txt
var filenamePattern = regexp.MustCompile(`^[A-Za-z0-9._-]+_[a-
        f0-9]{16}_\d{8}_\d{6}\.txt$`)

// CleanupOld removes persisted files older than maxAge from the
        base dir.
// Skips non-matching filenames (e.g., .gitkeep, README.md) and
        directories.
// Per-file errors are logged and the walk continues; the first
        error is
// returned for caller awareness but the sweep always completes.
func (m *Manager) CleanupOld(maxAge time.Duration) error {
    m.mu.Lock()
    defer m.mu.Unlock()

    entries, err := os.ReadDir(m.baseDir)
    if err != nil {
        if os.IsNotExist(err) {
            return nil
        }
        return fmt.Errorf("reading %s: %w", m.baseDir, err)
    }

    cutoff := time.Now().Add(-maxAge)
    var firstErr error
    for _, entry := range entries {
        if entry.IsDir() {
            continue
        }
        if !filenamePattern.MatchString(entry.Name()) {
            continue
        }
        info, statErr := entry.Info()
        if statErr != nil {
            log.Printf("WARN persistence: stat %s: %v",
        entry.Name(), statErr)
            if firstErr == nil {
                firstErr = statErr
            }
            continue
        }
        if info.ModTime().After(cutoff) {
            continue
        }
        path := filepath.Join(m.baseDir, entry.Name())
        if rmErr := os.Remove(path); rmErr != nil {
```

```
            log.Printf("WARN persistence: remove %s: %v", path,
        rmErr)
            if firstErr == nil {
                firstErr = rmErr
            }
        }
    }
    return firstErr
}
```

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run TestCleanupOld ./
        internal/tools/persistence/
```

Expected: PASS for 4 named tests.

- [ ] **Step 5: Run full package tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/tools/
        persistence/...
```

Expected: PASS — 8 + 4 + 4 = 16 tests.

- [ ] **Step 6: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/tools/persistence/ && echo "BLUFF
        FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/tools/persistence/manager.go
        helix_code/internal/tools/persistence/cleanup_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T05): CleanupOld with filename-pattern matching (TDD)

Manager.CleanupOld walks <baseDir> and removes only files
        matching the
```

```
canonical persistence pattern (<tool>_<hash16>_<timestamp>.txt)
        whose
ModTime is older than maxAge. Non-pattern files (.gitkeep,
        README.md)
and directories are left untouched. Missing baseDir is a no-op.
Per-file errors are logged and the walk completes; first error is
returned. 4 unit tests covering happy-path eviction, non-pattern
        leave-
alone, directory leave-alone, missing-base-dir no-op.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 6: Wire into internal/llm/tool_provider.go orchestration loop

Files: - Modify: `helix_code/internal/llm/tool_provider.go` - Create: `helix_code/internal/llm/tool_provider_persistence_test.go`

- [ ] **Step 1: Investigate current tool_provider.go**

Read the file to confirm `ToolCallingProvider` struct shape and where `executeToolCalls` is called:

```
grep -n 'type ToolCallingProvider\|executeToolCalls\|
        toolExecutor\|buildFinalPrompt' /run/media/milosvasic/
        DATA4TB/Projects/helix_code/helix_code/internal/llm/
        tool_provider.go
```

Confirm: - `ToolCallingProvider` struct exists. - `executeToolCalls(ctx, toolCalls) (map[string]interface{}, error)` exists around line 364. - `buildFinalPrompt` (line 413) consumes the result map and stringifies it.

The wiring strategy: add a `*persistence.Manager` field to `ToolCallingProvider`, plus a `SetPersistenceManager(m *persistence.Manager)` setter. After `executeToolCalls` returns, wrap each result through `MaybePersist` to produce a map of `*PersistedResult` instead of raw `interface{}`. `buildFinalPrompt` uses the wrapped result (inline `Output` if not persisted, the path-reference text if persisted).

- [ ] **Step 2: Write failing test**

Create `helix_code/internal/llm/tool_provider_persistence_test.go`:

```
package llm

import (
```

```
    "strings"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/tools/persistence"
)

func TestToolCallingProvider_SetsPersistenceManager(t
        *testing.T) {
    p := &ToolCallingProvider{}
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)
    p.SetPersistenceManager(m)
    assert.NotNil(t, p.persistenceManager)
}

func TestPersistResults_BelowThresholdInline(t *testing.T) {
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)
    p := &ToolCallingProvider{persistenceManager: m}

    results := map[string]interface{}{
        "Bash": "small output",
    }
    wrapped := p.persistResults(results)
    require.Len(t, wrapped, 1)
    assert.False(t, wrapped["Bash"].WasPersisted)
    assert.Equal(t, "small output", wrapped["Bash"].Output)
}

func TestPersistResults_AboveThresholdPersisted(t *testing.T) {
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)
    p := &ToolCallingProvider{persistenceManager: m}

    big := strings.Repeat("Z", persistence.PersistThreshold+10)
    results := map[string]interface{}{
        "Bash": big,
    }
    wrapped := p.persistResults(results)
    require.Len(t, wrapped, 1)
    assert.True(t, wrapped["Bash"].WasPersisted)
    assert.Empty(t, wrapped["Bash"].Output)
    assert.NotEmpty(t, wrapped["Bash"].PersistedOutputPath)
    assert.Equal(t, persistence.PersistThreshold+10,
        wrapped["Bash"].PersistedOutputSize)
```

```
}

func TestPersistResults_NilManagerPassthrough(t *testing.T) {
    p := &ToolCallingProvider{persistenceManager: nil}

    big := strings.Repeat("Z", persistence.PersistThreshold+10)
    results := map[string]interface{}{
        "Bash": big,
    }
    wrapped := p.persistResults(results)
    require.Len(t, wrapped, 1)
    assert.False(t, wrapped["Bash"].WasPersisted)
    assert.Equal(t, big, wrapped["Bash"].Output)
}

func TestPersistResults_NonStringResultStringified(t *testing.T)
        {
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)
    p := &ToolCallingProvider{persistenceManager: m}

    results := map[string]interface{}{
        "Calc": 42,
    }
    wrapped := p.persistResults(results)
    require.Len(t, wrapped, 1)
    assert.False(t, wrapped["Calc"].WasPersisted)
    assert.Equal(t, "42", wrapped["Calc"].Output)
}
```

- [ ] **Step 3: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run
        'TestToolCallingProvider_SetsPersistence|
        TestPersistResults' ./internal/llm/
```

Expected: FAIL — field and methods undefined.

- [ ] **Step 4: Modify tool_provider.go**

Add the import for `persistence`:

```
import (
    // existing imports
    "dev.helix.code/internal/tools/persistence"
)
```

Add the field to `ToolCallingProvider` struct (find the existing struct definition; add this field):

```
    persistenceManager *persistence.Manager
```

Add the setter and the wrap helper. Place these near `executeToolCalls`:

```
// SetPersistenceManager wires a persistence.Manager so that
        tool-result
// outputs above the threshold are written to disk and
        substituted with
// a path-reference in the final prompt. A nil manager disables
        persistence.
func (p *ToolCallingProvider) SetPersistenceManager(m
        *persistence.Manager) {
    p.persistenceManager = m
}

// persistResults wraps each tool result through MaybePersist.
        Non-string
// results are stringified via fmt.Sprintf("%v", v) before the
        size check.
func (p *ToolCallingProvider) persistResults(raw
        map[string]interface{})
        map[string]*persistence.PersistedResult {
    out := make(map[string]*persistence.PersistedResult,
        len(raw))
    for toolName, val := range raw {
        var s string
        switch v := val.(type) {
        case string:
            s = v
        default:
            s = fmt.Sprintf("%v", v)
        }
        res, err := p.persistenceManager.MaybePersist(toolName,
        "", s)
        if err != nil {
            // MaybePersist already falls back to inline on disk
        failure;
            // any error returned here is a programmer bug.
        Surface inline.
            res = &persistence.PersistedResult{Output: s,
        ToolName: toolName}
        }
        out[toolName] = res
    }
    return out
}
```

Modify `buildFinalPrompt` to accept the wrapped map. Replace the existing function (currently around line 413):

```
func (p *ToolCallingProvider) buildFinalPrompt(originalPrompt,
        initialResponse string, toolResults
        map[string]*persistence.PersistedResult) string {
    resultsStr := ""
    for toolName, result := range toolResults {
        if result.WasPersisted {
            resultsStr += fmt.Sprintf("- %s: [persisted to %s —
        %d chars. Use Read with that path to fetch full content.]
        \n",
                toolName, result.PersistedOutputPath,
        result.PersistedOutputSize)
        } else {
            resultsStr += fmt.Sprintf("- %s: %v\n", toolName,
        result.Output)
        }
    }

    return fmt.Sprintf(`Original request: %s

Initial response: %s

Tool execution results:
%s

Based on the tool results, provide your final answer:`,
        originalPrompt, initialResponse, resultsStr)
}
```

Find every callsite of `buildFinalPrompt` (search the file for `buildFinalPrompt(` invocations). Each callsite passes `toolResults` of type `map[string]interface{}` — change to call `p.persistResults(toolResults)` first:

```
// BEFORE:
//   finalPrompt := p.buildFinalPrompt(originalPrompt,
        initialResponse, toolResults)
// AFTER:
finalPrompt := p.buildFinalPrompt(originalPrompt,
        initialResponse, p.persistResults(toolResults))
```

- [ ] **Step 5: Run failing tests + new tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run
        'TestToolCallingProvider_SetsPersistence|
        TestPersistResults' ./internal/llm/
```

Expected: PASS for all 5 named tests.

- [ ] **Step 6: Run the full LLM package + persistence package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/llm/... ./internal/
        tools/persistence/...
```

Expected: PASS for both. If existing `internal/llm` tests fail because of the `buildFinalPrompt` signature change, fix the callers — `tool_provider_test.go` may need updates.

- [ ] **Step 7: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/tools/persistence/ internal/llm/
        tool_provider.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 8: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/llm/tool_provider.go helix_code/
        internal/llm/tool_provider_persistence_test.go
# If existing tool_provider_test.go was modified to fix call
        signatures:
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/llm/tool_provider_test.go 2>/dev/
        null || true
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T06): wire persistence.Manager into tool_provider
        orchestration

ToolCallingProvider now accepts a *persistence.Manager via
SetPersistenceManager; executeToolCalls' result map is wrapped
        via
persistResults before flowing into buildFinalPrompt. Persisted
        results
render as "[persisted to <path> — <size> chars]" in the prompt;
inline results render unchanged. 5 unit tests cover the wrap
        pipeline
including nil-manager passthrough and non-string stringification.
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 7: Audit + wire individual LLM providers

Files: - Modify (audit-driven): subset of `helix_code/internal/llm/{anthropic,azure,bedrock,copilot,gemini,groq,koboldai,llamacpp,local,ollam_provider.go` - Test: `helix_code/internal/llm/provider_persistence_audit_test.go`

This task EXAMINES every provider file to determine which ones independently handle `tool_result` content (and thus bypass `tool_provider.go`'s orchestration). Each non-conforming provider gets its own `MaybePersist` call.

- [ ] **Step 1: Run the audit grep**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
for f in internal/llm/*_provider.go; do
  case "$(basename "$f")" in
    tool_provider.go) continue;;
  esac
  hits=$(grep -c 'tool_result\|ToolResult\|toolResult' "$f")
  echo "$f: $hits"
done
```

Build a list of provider files with non-zero hits. For each, read the relevant section to determine: does it construct tool_result wire content directly, or does it call into `tool_provider.go`?

Document the audit result in the commit message.

- [ ] **Step 2: For each non-conforming provider, write a failing test**

Create `helix_code/internal/llm/provider_persistence_audit_test.go`:

```
package llm

import (
    "strings"
    "testing"

    "github.com/stretchr/testify/assert"

    "dev.helix.code/internal/tools/persistence"
)
```

```
// TestAllProvidersAcceptPersistenceManager is a contract test:
        every provider
// that handles tool_result must accept a *persistence.Manager
        for wrapping
// large outputs. The test fails if a provider doesn't implement
        the
// SetPersistenceManager hook (or an equivalent constructor
        option).
//
// Implementers: when adding a new provider that constructs
        tool_result
// content directly, add it to this test.
func TestAllProvidersAcceptPersistenceManager(t *testing.T) {
    // The reference implementation: ToolCallingProvider in
        tool_provider.go.
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)

    tcp := &ToolCallingProvider{}
    tcp.SetPersistenceManager(m)
    assert.NotNil(t, tcp.persistenceManager,
        "ToolCallingProvider must accept Manager")

    // Add a similar assertion here for any provider whose audit
        at T07 Step 1
    // showed it constructs tool_result wire content directly.
        The audit's
    // commit message lists the providers covered.
}

// TestPersistResults_ProducesExpectedRenderedString verifies the
        integration
// between persistResults and buildFinalPrompt: a persisted
        result must
// render as a path-reference, not as the original content.
func TestPersistResults_ProducesExpectedRenderedString(t
        *testing.T) {
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)
    tcp := &ToolCallingProvider{persistenceManager: m}

    big := strings.Repeat("X", persistence.PersistThreshold+1)
    wrapped := tcp.persistResults(map[string]interface{}{"Bash":
        big})
    rendered := tcp.buildFinalPrompt("orig", "init", wrapped)

    assert.Contains(t, rendered, "persisted to")
    assert.Contains(t, rendered, "Use Read with that path")
```

```
    // The original content should NOT appear in full; the path
        reference is
    // the entire visible payload from the tool result.
    assert.NotContains(t, rendered, big)
}
```

- [ ] **Step 3: Run the test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run
        'TestAllProvidersAcceptPersistenceManager|
        TestPersistResults_ProducesExpectedRenderedString' ./
        internal/llm/
```

Expected: PASS — both tests rely on `ToolCallingProvider` from T06.

- [ ] **Step 4: For each non-conforming provider identified in Step 1**

Wire `MaybePersist` directly. The pattern is:

1. Add a `persistenceManager *persistence.Manager` field to the provider struct.
2. Add a `SetPersistenceManager(m *persistence.Manager)` method.
3. Locate the function that constructs `tool_result` wire content (typically in the message-building or response-parsing path).
4. Before serialising the tool result, call `p.persistenceManager.MaybePersist(toolName, callID, output)` and use the returned `*PersistedResult` to choose between inline content and path-reference.
5. Add an assertion to `TestAllProvidersAcceptPersistenceManager` covering the new provider.

If the audit at Step 1 shows that NO provider bypasses `tool_provider.go` (likely — most delegate to `ToolCallingProvider`), the audit test alone is sufficient and no per-provider wiring is needed.

- [ ] **Step 5: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/llm/ && echo "BLUFF FOUND" || echo
        "clean"
```

Expected: `clean`.

- [ ] **Step 6: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/llm/
        provider_persistence_audit_test.go
# Plus any per-provider wiring identified at Step 4:
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/llm/<provider>_provider.go 2>/dev/
        null || true

git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T07): audit + wire individual LLM providers for
        persistence

Audit result (paste actual grep output from Step 1):
  internal/llm/anthropic_provider.go: <N> hits — <conforming|
        wired>
  internal/llm/azure_provider.go: <N> hits — <conforming|wired>
  internal/llm/bedrock_provider.go: <N> hits — <conforming|wired>
  internal/llm/copilot_provider.go: <N> hits — <conforming|wired>
  internal/llm/gemini_provider.go: <N> hits — <conforming|wired>
  internal/llm/groq_provider.go: <N> hits — <conforming|wired>
  internal/llm/koboldai_provider.go: <N> hits — <conforming|
        wired>
  internal/llm/llamacpp_provider.go: <N> hits — <conforming|
        wired>
  internal/llm/local_provider.go: <N> hits — <conforming|wired>
  internal/llm/ollama_provider.go: <N> hits — <conforming|wired>
  internal/llm/openai_compatible_provider.go: <N> hits —
        <conforming|wired>
  internal/llm/openai_provider.go: <N> hits — <conforming|wired>
  internal/llm/openrouter_provider.go: <N> hits — <conforming|
        wired>
  internal/llm/qwen_provider.go: <N> hits — <conforming|wired>
  internal/llm/vertexai_provider.go: <N> hits — <conforming|
        wired>
  internal/llm/xai_provider.go: <N> hits — <conforming|wired>

Adds contract test TestAllProvidersAcceptPersistenceManager and
        rendering
test TestPersistResults_ProducesExpectedRenderedString. Per-
        provider
wiring applied where audit found bypass paths.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

(Replace <N> and <conforming|wired> placeholders with the actual audit findings before committing.)

Task 8: System prompt note about persistedOutputPath

Files: - Modify: `helix_code/internal/agent/base_agent.go` (around line 596 — `getSystemPrompt`) - Create: `helix_code/internal/agent/system_prompt_persistence_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/agent/system_prompt_persistence_test.go`:

```
package agent

import (
    "strings"
    "testing"

    "github.com/stretchr/testify/assert"
)

func TestGetSystemPrompt_IncludesPersistedOutputNote(t
        *testing.T) {
    a := &BaseAgent{
        agentType:    "coding",
        name:         "test",
        capabilities: []string{"read", "write"},
    }
    prompt := a.getSystemPrompt()

    assert.Contains(t, prompt, "persistedOutputPath",
        "system prompt must teach the LLM about persisted
        outputs")
    assert.Contains(t, prompt, "Read",
        "system prompt must instruct the LLM to use the Read
        tool")
    assert.Contains(t, prompt, "50,000",
        "system prompt must reference the threshold so the LLM
        understands the trigger")
}

// Existing assertions remain — the prompt still describes the
        agent type/name.
func TestGetSystemPrompt_StillDescribesAgent(t *testing.T) {
    a := &BaseAgent{
        agentType:    "coding",
```

```
        name:         "test",
        capabilities: []string{"read"},
    }
    prompt := a.getSystemPrompt()
    assert.Contains(t, prompt, "coding")
    assert.Contains(t, prompt, "test")
    // Look for either a JSON instruction or a closing
        instruction
    assert.True(t, strings.Contains(prompt, "JSON") ||
        strings.Contains(prompt, "Read"))
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run
        TestGetSystemPrompt_IncludesPersistedOutputNote ./
        internal/agent/
```

Expected: FAIL — prompt does not yet mention `persistedOutputPath`.

- [ ] **Step 3: Modify base_agent.go's getSystemPrompt**

Update `helix_code/internal/agent/base_agent.go` `getSystemPrompt` (around line 596) to:

```
// getSystemPrompt returns the system prompt for the agent
func (a *BaseAgent) getSystemPrompt() string {
    return fmt.Sprintf(`You are a %s agent named %s. Your
        capabilities include: %v.

You are part of a multi-agent system for software development.
        Your responses should be:
1. Precise and actionable
2. Formatted as JSON as requested
3. Focused on your area of expertise

Tool output handling: when a tool produces output exceeding
        50,000 characters, the runtime persists the raw content
        to disk. The tool result you receive will contain a
        "persistedOutputPath" pointing to a file under .helix/
        tool-results/. To read the full content, invoke the Read
        tool with that path. Treat the path as a regular
        workspace file.
```

```
Always respond with valid JSON only, no additional text.`,
        a.agentType, a.name, a.capabilities)
}
```

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run TestGetSystemPrompt ./
        internal/agent/
```

Expected: PASS for both tests.

- [ ] **Step 5: Run the whole agent package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/agent/...
```

Expected: PASS — confirm no regression in other agent tests.

- [ ] **Step 6: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/agent/base_agent.go && echo "BLUFF
        FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/agent/base_agent.go helix_code/
        internal/agent/system_prompt_persistence_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T08): system prompt note about persistedOutputPath

BaseAgent.getSystemPrompt now teaches the LLM that tool outputs
        over
50,000 characters are persisted to disk and that
        "persistedOutputPath"
in the result indicates where to find the full content (read with
        the
existing Read tool). 2 unit tests verify the prompt contains the
required keywords + still describes the agent.
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 9: cmd/cli/main.go startup + integration test (no mocks)

Files: - Modify: `helix_code/cmd/cli/main.go` - Test: `helix_code/tests/integration/persistence/persistence_integration_test.go`

- [ ] **Step 1: Investigate cmd/cli/main.go's CLI struct**

```
grep -n 'type CLI\|type cli\|func.*\*CLI.*Run\|
        persistenceManager\|permissionsEngine' /run/media/
        milosvasic/DATA4TB/Projects/helix_code/helix_code/cmd/
        cli/main.go | head -10
```

Confirm where the CLI struct + bootstrap path live. From F02 we know the CLI is `flag`-based with `(*CLI).Run()`. The persistence wiring follows the same pattern.

- [ ] **Step 2: Add persistence bootstrap to main.go**

Add the import:

```
import (
    // existing imports
    "dev.helix.code/internal/tools/persistence"
)
```

Add the field to the CLI struct (find the struct that already holds `permissionMode` and `permissionsEngine` from F02):

```
    persistenceManager *persistence.Manager
```

Add a method to construct + wire the manager. Place near `initPermissions`:

```
func (c *CLI) initPersistence() error {
    cwd, err := os.Getwd()
    if err != nil {
        return fmt.Errorf("resolving cwd for persistence: %w",
        err)
    }
    c.persistenceManager = persistence.NewManager(cwd)
    go func() {
        if err :=
        c.persistenceManager.CleanupOld(persistence.DefaultMaxAge);
        err != nil {
            log.Printf("WARN persistence cleanup: %v", err)
```

```
        }
    }()
    return nil
}
```

Call it from the existing CLI startup sequence (find the line that calls `c.initPermissions(...)` and add `c.initPersistence()` adjacent to it):

```
if err := c.initPermissions(ctx, policyEngine); err != nil {
    return fmt.Errorf("permissions init: %w", err)
}
if err := c.initPersistence(); err != nil {
    return fmt.Errorf("persistence init: %w", err)
}
```

(The persistence manager is plumbed into LLM providers in T07; T09 only constructs it. Future task T11 close-out documents the gap if no production LLM provider has yet been retrofitted to receive the manager.)

- [ ] **Step 3: Verify it compiles**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go build ./cmd/cli/...
```

Expected: clean compile.

- [ ] **Step 4: Write the integration test**

Create the directory and test file:

```
mkdir -p /run/media/milosvasic/DATA4TB/Projects/helix_code/
        helix_code/tests/integration/persistence
```

Create `helix_code/tests/integration/persistence/persistence_integration_test.go`:

```
//go:build integration

package persistence_test

import (
    "os"
    "path/filepath"
    "strings"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
```

```
    "dev.helix.code/internal/tools/persistence"
)

// TestIntegration_LargeOutputIsPersistedAndReloadable proves
        that a >50K
// output is written to disk under .helix/tool-results/ and the
        resulting
// PersistedResult.PersistedOutputPath is a real file whose
        content matches
// the original byte-for-byte. NO mocks.
func TestIntegration_LargeOutputIsPersistedAndReloadable(t
        *testing.T) {
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)

    // Real input: 60_000 X bytes (well above 50K threshold).
    output := strings.Repeat("X", 60_000)
    res, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err)
    require.True(t, res.WasPersisted)

    // PROOF 1: file exists at the reported path.
    info, err := os.Stat(res.PersistedOutputPath)
    require.NoError(t, err)
    assert.False(t, info.IsDir())
    assert.Equal(t, int64(60_000), info.Size())

    // PROOF 2: file content matches the original.
    body, err := os.ReadFile(res.PersistedOutputPath)
    require.NoError(t, err)
    assert.Equal(t, output, string(body))

    // PROOF 3: LoadPersisted returns the same content.
    loaded, err := m.LoadPersisted(res.PersistedOutputPath)
    require.NoError(t, err)
    assert.Equal(t, output, loaded)

    // PROOF 4: the file lives under the expected baseDir.
    expectedDir := filepath.Join(tmp, persistence.PersistDir)
    assert.Equal(t, expectedDir,
        filepath.Dir(res.PersistedOutputPath))
}

// TestIntegration_BelowThresholdNeverWritesToDisk proves that a
        small
// output never creates a file under .helix/tool-results/.
func TestIntegration_BelowThresholdNeverWritesToDisk(t
        *testing.T) {
```

```
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)

    output := strings.Repeat("Y", persistence.PersistThreshold-1)
    res, err := m.MaybePersist("Bash", "call-1", output)
    require.NoError(t, err)
    require.False(t, res.WasPersisted)

    // The persistence dir should not have been created.
    _, err = os.Stat(filepath.Join(tmp, persistence.PersistDir))
    assert.True(t, os.IsNotExist(err),
        "below-threshold persist must not create the .helix/tool-
        results dir")
}

// TestIntegration_PathTraversalIsRejected proves that
        LoadPersisted refuses
// to read files outside its baseDir, even if a malicious path is
        provided.
func TestIntegration_PathTraversalIsRejected(t *testing.T) {
    tmp := t.TempDir()
    m := persistence.NewManager(tmp)

    // Create a sensitive file outside the baseDir
    sensitive := filepath.Join(tmp, "secrets.txt")
    require.NoError(t, os.WriteFile(sensitive,
        []byte("topsecret"), 0o600))

    // Try to load it via the absolute path
    _, err := m.LoadPersisted(sensitive)
    assert.Error(t, err)
    assert.Contains(t, err.Error(), "path outside persistence
        directory")
}
```

- [ ] **Step 5: Run integration tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -tags=integration ./tests/
        integration/persistence/...
```

Expected: PASS for 3 tests.

- [ ] **Step 6: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" cmd/cli/main.go internal/tools/persistence/
        tests/integration/persistence/ && echo "BLUFF FOUND" ||
        echo "clean"
```

Expected: `clean`.

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/cmd/cli/main.go helix_code/tests/integration/
        persistence/
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T09): cmd/cli/main.go startup + integration tests (no
        mocks)

CLI bootstrap constructs persistence.Manager rooted at os.Getwd()
        and
spawns a background CleanupOld(7d) goroutine. Three integration
        tests
with -tags=integration and NO mocks: large output is persisted +
reloadable, below-threshold never creates the dir, path-traversal
        is
rejected.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 10: Challenge with three runtime-evidence scenarios

Files: - Create: `helix_code/tests/e2e/challenges/persistence/expected.json` - Create: `helix_code/tests/e2e/challenges/persistence/run.sh` - Create: `helix_code/tests/e2e/challenges/persistence/README.md`

- [ ] **Step 1: Create the directory**

```
mkdir -p /run/media/milosvasic/DATA4TB/Projects/helix_code/
        helix_code/tests/e2e/challenges/persistence
```

- [ ] **Step 2: Write expected.json**

Create `helix_code/tests/e2e/challenges/persistence/expected.json`:

```
{
  "name": "persistence/tool-result-end-to-end",
  "feature": "P1-F03 — Tool Result Persistence",
  "scenarios": [
    {
      "id": "S1-below-threshold-inline",
      "input_size": 49999,
      "expected_was_persisted": false,
      "expected_dir_exists_after": false
    },
    {
      "id": "S2-above-threshold-persisted",
      "input_size": 50001,
      "expected_was_persisted": true,
      "expected_file_byte_count": 50001
    },
    {
      "id": "S3-hash-idempotence-same-filename",
      "input_size": 60000,
      "expected_first_filename_equals_second_filename": true,
      "note":
        "two persists of identical content must produce the same
        file (hash-collision filename)"
    }
  ]
}
```

☐

Step 3: Write run.sh

Create `helix_code/tests/e2e/challenges/persistence/run.sh`:

```
#!/usr/bin/env bash
# Challenge: P1-F03 — Tool Result Persistence end-to-end runtime
        evidence.
# Drives the persistence.Manager directly through a Go test
        binary that
# emits machine-readable evidence (the persisted path + file
        size) to stdout.
set -euo pipefail

HERE=$(cd "$(dirname "$0")" && pwd)
ROOT=$(cd "$HERE/../../../.." && pwd)
WORK=$(mktemp -d)
trap 'rm -rf "$WORK"' EXIT

# Build a tiny Go driver that exercises persistence.Manager.
```

```
cat > "$WORK/driver.go" <<'EOF'
package main

import (
    "fmt"
    "os"
    "path/filepath"
    "strings"

    "dev.helix.code/internal/tools/persistence"
)

func main() {
    if len(os.Args) < 4 {
        fmt.Fprintln(os.Stderr, "usage: driver <projectRoot>
        <scenario> <inputSize>")
        os.Exit(2)
    }
    projectRoot := os.Args[1]
    scenario := os.Args[2]
    var size int
    if _, err := fmt.Sscanf(os.Args[3], "%d", &size); err != nil
        {
        fmt.Fprintf(os.Stderr, "bad size: %v\n", err)
        os.Exit(2)
    }

    m := persistence.NewManager(projectRoot)
    output := strings.Repeat("X", size)

    switch scenario {
    case "single":
        res, err := m.MaybePersist("Bash", "call-1", output)
        if err != nil {
            fmt.Fprintf(os.Stderr, "MaybePersist error: %v\n",
        err)
            os.Exit(1)
        }
        fmt.Printf("was_persisted=%v\n", res.WasPersisted)
        fmt.Printf("path=%s\n", res.PersistedOutputPath)
        fmt.Printf("size=%d\n", res.PersistedOutputSize)
        fmt.Printf("dir_exists=%v\n",
        dirExists(filepath.Join(projectRoot,
        persistence.PersistDir)))
    case "twice":
        r1, err := m.MaybePersist("Bash", "call-1", output)
        if err != nil {
```

```
            fmt.Fprintf(os.Stderr, "first MaybePersist error:
        %v\n", err)
            os.Exit(1)
        }
        r2, err := m.MaybePersist("Bash", "call-2", output)
        if err != nil {
            fmt.Fprintf(os.Stderr, "second MaybePersist error:
        %v\n", err)
            os.Exit(1)
        }
        fmt.Printf("first_path=%s\n", r1.PersistedOutputPath)
        fmt.Printf("second_path=%s\n", r2.PersistedOutputPath)
        // Hash idempotence: same content → filenames share the
        hash prefix
        // (timestamps may differ at second-granularity).
        base1 := filepath.Base(r1.PersistedOutputPath)
        base2 := filepath.Base(r2.PersistedOutputPath)
        // Filename layout: <tool>_<hash16>_<timestamp>.txt →
        split on '_'
        parts1 := strings.SplitN(base1, "_", 3)
        parts2 := strings.SplitN(base2, "_", 3)
        hashesMatch := len(parts1) >= 2 && len(parts2) >= 2 &&
        parts1[1] == parts2[1]
        fmt.Printf("hashes_match=%v\n", hashesMatch)
    default:
        fmt.Fprintf(os.Stderr, "unknown scenario %q\n", scenario)
        os.Exit(2)
    }
}

func dirExists(p string) bool {
    info, err := os.Stat(p)
    return err == nil && info.IsDir()
}
EOF

DRIVER_BIN="$WORK/driver"
(cd "$ROOT" && go build -o "$DRIVER_BIN" "$WORK/driver.go")

# Scenario 1: below threshold → inline, dir not created
echo "=== S1: below-threshold inline ==="
S1_ROOT="$WORK/s1"
mkdir -p "$S1_ROOT"
S1_OUT=$("$DRIVER_BIN" "$S1_ROOT" single 49999)
echo "$S1_OUT"
if ! echo "$S1_OUT" | grep -q "^was_persisted=false$"; then
  echo "FAIL S1: expected was_persisted=false"
  exit 1
```

```
fi
if ! echo "$S1_OUT" | grep -q "^dir_exists=false$"; then
  echo "FAIL S1: persistence dir was created for below-threshold
        output"
  exit 1
fi

# Scenario 2: above threshold → persisted, file exists, byte
        count matches
echo
echo "=== S2: above-threshold persisted ==="
S2_ROOT="$WORK/s2"
mkdir -p "$S2_ROOT"
S2_OUT=$("$DRIVER_BIN" "$S2_ROOT" single 50001)
echo "$S2_OUT"
if ! echo "$S2_OUT" | grep -q "^was_persisted=true$"; then
  echo "FAIL S2: expected was_persisted=true"
  exit 1
fi
S2_PATH=$(echo "$S2_OUT" | grep '^path=' | sed 's/^path=//')
if [[ ! -f "$S2_PATH" ]]; then
  echo "FAIL S2: file not at $S2_PATH"
  exit 1
fi
S2_BYTES=$(wc -c < "$S2_PATH" | tr -d ' ')
if [[ "$S2_BYTES" != "50001" ]]; then
  echo "FAIL S2: byte count $S2_BYTES != 50001"
  exit 1
fi

# Scenario 3: hash idempotence — same content twice → same hash
        prefix
echo
echo "=== S3: hash idempotence ==="
S3_ROOT="$WORK/s3"
mkdir -p "$S3_ROOT"
S3_OUT=$("$DRIVER_BIN" "$S3_ROOT" twice 60000)
echo "$S3_OUT"
if ! echo "$S3_OUT" | grep -q "^hashes_match=true$"; then
  echo "FAIL S3: identical content produced different hash
        filenames"
  exit 1
fi

echo
echo "PASS: all three scenarios produced expected outcomes"
```

Step 4: Make it executable

```
chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
        helix_code/tests/e2e/challenges/persistence/run.sh
```

☐

Step 5: Write README.md

Create `helix_code/tests/e2e/challenges/persistence/README.md`:

```
# Challenge — Tool Result Persistence (P1-F03)

End-to-end runtime evidence that the persistence layer's
        behaviour
matches the spec for three boundary scenarios.

## Scenarios

1. **S1 — below threshold (49,999 bytes)**: `MaybePersist`
        returns inline; `.helix/tool-results/` is not created.
2. **S2 — above threshold (50,001 bytes)**: persisted; the file
        exists at the reported path; `wc -c` matches 50,001.
3. **S3 — hash idempotence**: two persists of identical 60,000-
        byte content produce filenames that share the same
        `sha256[:16]` hash prefix.

## Run

```bash
cd HelixCode && tests/e2e/challenges/persistence/run.sh
```

Exit 0 means PASS. Exit non-zero means at least one scenario failed.

Mutation test (CONST-039)

To verify the Challenge actually catches a broken engine:

```
// in internal/tools/persistence/types.go:
const PersistThreshold = 0  // <-- mutation: every output
        persists
```

Re-run `run.sh`. S1 MUST FAIL because every output now triggers persistence (`was_persisted=true` instead of `false`). Revert the mutation and confirm PASS.

- [ ] **Step 6: Run the Challenge**

```bash
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode && tests/e2
```
```

Expected: PASS at the end. Exit 0.

- [ ] **Step 7: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" tests/e2e/challenges/persistence/ && echo
        "BLUFF FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 8: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/tests/e2e/challenges/persistence/
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F03-T10): Challenge for tool-result persistence with
        runtime evidence

Three scenarios driven by a Go-built driver invoking
        persistence.Manager
directly:
  S1: below-threshold (49,999 bytes) → inline; dir never created.
  S2: above-threshold (50,001 bytes) → persisted; wc -c matches.
  S3: identical-content twice → filenames share sha256[:16]
        prefix
      (hash idempotence).

Mutation-test recipe in README.md ensures the Challenge will FAIL
        if
PersistThreshold is set to 0.

Runtime evidence: see commit body of P1-F03-T11 close-out.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 11: Feature 3 close-out + push

Files: - Modify: `docs/improvements/06_phase_1_evidence.md` - Modify: `docs/improvements/PROGRESS.md`

- [ ] **Step 1: Re-run the Challenge to capture fresh evidence**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && tests/e2e/challenges/persistence/run.sh 2>&1 | tee /
        tmp/p1-f03-t11-rerun.txt
```

Expected: PASS. If FAIL, STOP and investigate.

- [ ] **Step 2: Run final regression test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && \
  go test -count=1 -race ./internal/tools/persistence/... ./
        internal/tools/permissions/... ./internal/tools/
        confirmation/... ./internal/llm/... ./internal/
        agent/... ./cmd/cli/... 2>&1 | tee /tmp/p1-f03-t11-
        tests.txt && \
  go test -count=1 -race -tags=integration ./tests/integration/
        persistence/... ./tests/integration/permissions/... 2>&1
        | tee -a /tmp/p1-f03-t11-tests.txt
```

Expected: PASS for every package.

- [ ] **Step 3: Run verify-foundation gate**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode && make
        verify-foundation 2>&1 | tee /tmp/p1-f03-t11-verify.txt
```

The LLMsVerifier dual-pin parking lot from Phase 0 may cause exit 2 (same baseline as F01 + F02). Capture the full output verbatim.

- [ ] **Step 4: Anti-bluff smoke (broad)**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/tools/persistence/ tests/e2e/challenges/persistence/ \
  tests/integration/persistence/ internal/llm/tool_provider.go \
  internal/llm/tool_provider_persistence_test.go \
  internal/llm/provider_persistence_audit_test.go \
  internal/agent/system_prompt_persistence_test.go && echo
        "BLUFF FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 5: Append runtime evidence to evidence file**

In `docs/improvements/06_phase_1_evidence.md`, replace the F03 `### Task evidence trail` placeholder with:

```
### Task evidence trail

- T01 — `<sha-T01>` — bootstrap evidence + advance PROGRESS
- T02 — `<sha-T02>` — persistence package skeleton
- T03 — `<sha-T03>` — Manager.MaybePersist (8 unit tests)
- T04 — `<sha-T04>` — LoadPersisted with path-traversal guard (4
        unit tests)
- T05 — `<sha-T05>` — CleanupOld with filename-pattern matching
        (4 unit tests)
- T06 — `<sha-T06>` — wire into tool_provider orchestration (5
        unit tests)
- T07 — `<sha-T07>` — audit + wire individual LLM providers
- T08 — `<sha-T08>` — system prompt note about
        persistedOutputPath (2 unit tests)
- T09 — `<sha-
        T09>` — cmd/cli/main.go startup + integration tests (3
        tests, no mocks)
- T10 — `<sha-T10>` — Challenge with runtime evidence (3
        scenarios)

### Challenge runtime evidence (from T10, re-verified at T11
        close-out)
```

<paste verbatim contents of /tmp/p1-f03-t11-rerun.txt — full S1/S2/S3 transcript>

```
### Anti-bluff scan
```

<paste actual command + ‘clean’ output from Step 4>

```
### Verify-foundation gate
```

<paste verbatim contents of /tmp/p1-f03-t11-verify.txt>

```
### Closure

F03 closed 2026-05-05. F04 (Git Worktree Agent Isolation) unblocked.
```

Replace `<sha-TNN>` placeholders with actual short SHAs from `git log --oneline -16`.

- [ ] **Step 6: Update PROGRESS.md**

Edit `docs/improvements/PROGRESS.md`:

1. Update the Current focus block:

```
## Current focus
- **Active phase:** P1 — claude-code feature porting
```

```
- **Active feature:** F04 — Git Worktree Agent Isolation (awaits
      its own writing-plans cycle)
- **Active task:** pending
- **Last completed:** P1-F03-T11 — Feature 3 (Tool Result
      Persistence) close-out + push
- **Owner:** agent (Claude Opus 4.7)
- **Started:** 2026-05-04
- **Last touched:** 2026-05-05
- **Blocked-on:** none
```

1. Mark every F03 task `[x]` in the F03 task list block (T01 through T11).

2. Append a Decision-log entry:

```
-
      2026-05-05 — Feature 3 (Tool Result Persistence) closed.
      Eleven sub-commits. New thin sub-package `internal/tools/
      persistence` mirrors F02's pattern. Threshold check fires
      at the LLM provider boundary (tool_provider.go), so non-
      LLM callers stay inline. LLM reads back via the existing
      Read tool — no new tool added. Lazy 7-day CleanupOld at
      startup. Engine proven via 3 integration tests + 3
      Challenge scenarios (above/below threshold + hash
      idempotence).
```

1. Append a parking-lot entry (if T07's audit found any provider that requires per-provider wiring follow-up):

```
- **LLM provider per-file persistence wiring (from P1-F03-T07
      audit):** if T07's audit revealed that any of the 16 LLM
      providers bypasses tool_provider.go's orchestration and
      assembles tool_result content directly, those providers'
      wiring is documented in T07's commit message. Any
      incomplete wiring is a Phase 3 follow-up — the engine
      works correctly via the orchestration boundary; per-
      provider wiring is a defense-in-depth measure for
      providers that don't go through the loop.
```

(If T07's audit found ALL providers conform to `tool_provider.go`, omit this parking-lot entry — there's nothing to track.)

- [ ] **Step 7: Commit close-out**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
      improvements/06_phase_1_evidence.md docs/improvements/
      PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
      -m "$(cat <<'EOF'
chore(P1-F03-T11): Feature 3 (Tool Result Persistence) close-out
```

```
Eleven sub-commits. New internal/tools/persistence sub-package
        owns
blob storage at <projectRoot>/.helix/tool-results/ with a 50,000-
        byte
threshold, sha256-hashed filenames, path-traversal-guarded
        LoadPersisted,
and a 7-day age-based CleanupOld. Threshold check fires at the
        LLM
provider boundary (tool_provider.go), so non-LLM callers stay
        inline.
LLM reads back persisted blobs via the existing Read tool; system
prompt teaches the convention.

Challenge runtime evidence (verbatim from tests/e2e/challenges/
        persistence/run.sh):

<paste full S1/S2/S3 transcript from /tmp/p1-f03-t11-rerun.txt>

Anti-bluff scan: clean.
Verify-foundation gate: <exit 0 OR same Phase 0 LLMsVerifier-pin
        baseline>.

PROGRESS advanced: F03 done; F04 (Git Worktree Agent Isolation)
        unblocked.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

- [ ] **Step 8: Push to all four configured remotes (NON-FORCE per CONST-043)**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        origin main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        github main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        gitlab main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        upstream main
```

If any push fails non-fast-forward, STOP and report — do NOT use `--force`. The remote being ahead means manual reconciliation is needed.

- [ ] **Step 9: Verify upstream parity**

```
HEAD=$(git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode
        rev-parse HEAD)
echo "HEAD: $HEAD"
for r in origin github gitlab upstream; do
  echo "=== $r ==="
  git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode ls-
        remote --heads "$r" main
done
```

Expected: all four remotes' `main` SHA equals HEAD.

Self-review against the spec

Walked spec section-by-section against the plan:

- **§1.4 S1 (`make verify-compile` exits 0)** — covered by T02 step 3, T06 step 6, T11 step 2.
- **§1.4 S2 (unit tests with `-race`)** — every TDD task uses `-race`.
- **§1.4 S3 (integration test, no mocks)** — T09.
- **§1.4 S4 (Challenge + runtime evidence pasted)** — T10 + T11.
- **§1.4 S5 (anti-bluff smoke clean)** — every task ends with the smoke check.
- **§1.4 S6 (CleanupOld pattern matching)** — T05 has the pattern-match test.
- **§1.4 S7 (path-traversal guard)** — T04 has explicit traversal tests.
- **§2.3 component table** — every entry maps to T02–T09.
- **§3.1 constants + §3.2 PersistedResult** — T02.
- **§3.3 Manager API** — T03 (NewManager + MaybePersist), T04 (LoadPersisted), T05 (CleanupOld).
- **§3.4 filename pattern** — T03 implements; T05 matches against it.
- **§4 threshold semantics** — T03 unit tests cover boundary (49,999 / 50,000 / 50,001).
- **§5 read-back path** — T08 system prompt note.
- **§6 cleanup semantics** — T05.
- **§7 error handling** — T03 disk-full test, T04 traversal/missing-file tests, T05 per-file-error logging, T03 nil-Manager pass-through.
- **§8.5 mutation test** — T10 README.md documents it.

No spec section is uncovered.

Type consistency: `Manager`, `NewManager`, `MaybePersist`, `LoadPersisted`, `CleanupOld`, `PersistedResult`, `PersistThreshold`, `PersistDir`, `DefaultMaxAge`, `ErrPathTraversal` — all referenced consistently across tasks.

Placeholder scan: every step has either real code, a real command, or a real verification check. The literal `<sha-TNN>` strings in T11 step 5 are intentional fillers (cannot be known until the prior commits land). The `<N>` and `<conforming|wired>` placeholders in T07's commit message are deliberate — they're filled in from the actual audit at commit time.

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-05-p1-f03-tool-result-persistence.md`. Two execution options:

1. Subagent-Driven (recommended) — fresh subagent per task, review between tasks, fast iteration.

2. Inline Execution — execute tasks in this session using executing-plans, batch execution with checkpoints.

Which approach?